



NOMBRE:

MINAYA GARCIA JEFFERSON CRISTOPHER

CARRERA:

INGENIERIA EN TECNOLOGIAS DE LA INFORMACION

NIVEL:

Nº 2

2025

Introducción

La concurrencia en sistemas operativos se refiere a la capacidad de ejecutar múltiples procesos o hilos de forma simultánea o intercalada, lo que permite un uso más eficiente de los recursos del sistema y mejora la capacidad de respuesta. Sin embargo, la ejecución concurrente introduce desafíos significativos, principalmente cuando los procesos o hilos comparten recursos. Para gestionar estos desafíos y asegurar la integridad de los datos, se emplean mecanismos de sincronización como los mutex, las barreras y los eventos [1], [2].

Esta actividad se centra en la explicación detallada de ejemplos prácticos en Python que ilustran el uso de threading.Lock (mutex), threading.Barrier (barreras) y threading.Event (eventos) para la sincronización de hilos, así como en la identificación de los problemas comunes que estos mecanismos ayudan a evitar.

1. Explicación Detallada del Código y Funcionamiento de threading.Lock (Mutex)

El primer ejemplo proporcionado utiliza threading.Lock para implementar un mecanismo de exclusión mutua, comúnmente conocido como mutex. Un mutex es una primitiva de sincronización que garantiza que solo un hilo pueda acceder a una sección crítica de código a la vez, previniendo así las condiciones de carrera (race conditions) [3].

```
import threading

# Definimos una variable global compartida
contador_global = 0

# Creamos un objeto mutex
mutex = threading.Lock()

# Función que incrementa el contador global de forma segura utilizando un mutex
def incrementar():
    global contador_global
    # Adquirimos el mutex
    mutex.acquire()
    try:
        # Sección crítica: Incrementamos el contador
        contador_global += 1
    finally:
        # Liberamos el mutex
        mutex.release()

# Función que ejecuta la tarea de incrementar el contador un número
# determinado de veces
def tarea():
    for _ in range(100000):
        incrementar()

# Creamos dos hilos que ejecutarán la misma tarea
hilo1 = threading.Thread(target=tarea)
hilo2 = threading.Thread(target=tarea)

# Iniciamos los hilos
hilo1.start()
hilo2.start()

# Esperamos a que ambos hilos terminen
hilo1.join()
hilo2.join()

# Imprimimos el valor final del contador global

print("El valor final del contador global es:", contador_global)
```

Funcionamiento:

- 1 **contador_global = 0**: Se inicializa una variable global que será compartida y modificada por múltiples hilos. Sin sincronización, esta variable sería susceptible a condiciones de carrera.
- 2 **mutex = threading.Lock()**: Se crea una instancia de Lock, que actúa como el mecanismo de exclusión mutua. Un Lock tiene dos estados: bloqueado y desbloqueado. Inicialmente, está desbloqueado.
- 3 **incrementar()** función: Esta función es la sección crítica. Antes de modificar contador_global, el hilo llama a mutex.acquire() para intentar adquirir el bloqueo. Si el bloqueo ya está en posesión de otro hilo, el hilo actual se bloqueará hasta que el bloqueo sea liberado. Una vez que el bloqueo es adquirido, el hilo entra en la sección crítica y puede modificar contador_global de forma segura. El bloque try...finally asegura que mutex.release() sea llamado para liberar el bloqueo, incluso si ocurre una excepción dentro de la sección crítica. Esto es crucial para evitar interbloqueos (deadlocks).
- 4 **tarea()** función: Esta función simula una carga de trabajo, llamando a incrementar() 100,000 veces. Cada llamada a incrementar() intentará adquirir y liberar el mutex.
- 5 Creación e inicio de hilos: Se crean dos hilos (hilo1, hilo2) que ejecutarán la función tarea(). Ambos hilos se inician con start().
- 6 **join()**: hilo1.join() y hilo2.join() aseguran que el programa principal espere a que ambos hilos terminen su ejecución antes de imprimir el valor final de contador_global. Esto garantiza que todas las operaciones de incremento se hayan completado.

Problemas que evita el Mutex:

El uso de un mutex en este escenario evita principalmente las condiciones de carrera (race conditions). Una condición de carrera ocurre cuando múltiples hilos acceden y modifican datos compartidos concurrentemente, y el resultado final de la ejecución depende del orden en que se ejecutan las operaciones. Sin el mutex, el contador_global podría no reflejar el valor esperado (200,000 en este caso), ya que las operaciones de lectura, modificación y escritura del contador por parte de los hilos podrían intercalarse de manera impredecible, llevando a pérdidas de actualizaciones [3], [4].

2. Explicación Detallada del Código y Funcionamiento de threading.Barrier (Barreras)

El segundo ejemplo ilustra el uso de `threading.Barrier`, un mecanismo de sincronización que permite a un número fijo de hilos esperar hasta que todos hayan alcanzado un punto común (la barrera) antes de continuar su ejecución [1].

```
import threading

# Creamos una barrera para sincronizar dos hilos
barrera = threading.Barrier(2)

# Función que ejecuta la tarea de imprimir un mensaje y esperar en la barrera
def tarea():
    print("Hilo iniciado")
    # Esperamos en la barrera
    barrera.wait()
    print("Hilo continuando")

# Creamos dos hilos que ejecutarán la misma tarea
hilo1 = threading.Thread(target=tarea)
hilo2 = threading.Thread(target=tarea)

# Iniciamos los hilos
hilo1.start()
hilo2.start()

# Esperamos a que ambos hilos terminen
hilo1.join()
hilo2.join()

print("Programa terminado")
```

Funcionamiento:

- 1 **barrera = threading.Barrier(2)**: Se crea una barrera que requiere que dos hilos (parties=2) lleguen a ella antes de que cualquiera pueda continuar. El número de parties debe coincidir con el número de hilos que se espera que lleguen a la barrera.
- 2 **tarea()** función: Cada hilo que ejecuta tarea() imprimirá "Hilo iniciado" y luego llamará a barrera.wait(). Esta llamada bloqueará al hilo hasta que el número especificado de hilos (en este caso, 2) haya llegado a la barrera. Una vez que ambos hilos han llegado, la barrera se "rompe" y todos los hilos bloqueados son liberados para continuar su ejecución, imprimiendo "Hilo continuando".
- 3 Creación e inicio de hilos: Similar al ejemplo anterior, se crean y se inician dos hilos que ejecutarán la función tarea().
- 4 **join()**: Se espera a que ambos hilos finalicen antes de imprimir "Programa terminado".

Problemas que evita la Barrera:

Las barreras son útiles para la sincronización de fases en algoritmos paralelos. Evitan que un hilo avance a la siguiente fase de computación antes de que todos los demás hilos hayan completado la fase actual. Esto previene problemas donde los resultados de una fase dependen de la finalización de todos los participantes, asegurando que todos los hilos estén en un estado consistente antes de proceder [1], [5].

3. Explicación Detallada del Código y Funcionamiento de threading.Event (Eventos)

El tercer ejemplo demuestra el uso de `threading.Event`, un mecanismo de sincronización que permite a un hilo señalar un evento y a otros hilos esperar hasta que ese evento sea señalado [1].

```
import threading

import time

# Creamos un objeto evento
evento = threading.Event()

# Función que espera a que se active el evento
def esperar_evento():
    print("Esperando al evento...")
    # Esperamos a que el evento se active
    evento.wait()
    print("El evento ha sido activado!")

# Función que activa el evento después de un cierto tiempo
def activar_evento():
    print("Esperando 5 segundos antes de activar el evento...")
    time.sleep(5)
    # Activamos el evento
    evento.set()
    print("El evento ha sido activado después de 5 segundos")

# Creamos dos hilos que ejecutarán las funciones
hilo1 = threading.Thread(target=esperar_evento)
hilo2 = threading.Thread(target=activar_evento)

# Iniciamos los hilos
hilo1.start()
hilo2.start()

# Esperamos a que ambos hilos terminen
hilo1.join()
hilo2.join()

print("Programa terminado")
```

Funcionamiento:

- 1 **evento = threading.Event()**: Se crea una instancia de Event. Un objeto Event mantiene una bandera interna que puede ser True o False. Inicialmente, la bandera es False.
- 2 **esperar evento()** función: Este hilo llama a evento.wait(). Si la bandera interna del evento es False, este hilo se bloqueará hasta que la bandera se establezca en True mediante una llamada a evento.set(). Una vez que la bandera es True, el hilo continúa su ejecución.
- 3 **activar evento()** función: Este hilo simula una tarea que toma tiempo (time.sleep(5)). Después de 5 segundos, llama a evento.set(), lo que cambia la bandera interna del evento a True. Esto despierta a cualquier hilo que esté esperando en evento.wait().
- 4 Creación e inicio de hilos: Se crean y se inician dos hilos, uno para esperar evento() y otro para activar evento().
- 5 **join()**: Se espera a que ambos hilos finalicen antes de imprimir "Programa terminado".

Problemas que evita el Evento:

Los eventos son ideales para la comunicación entre hilos basada en señales. Permiten que un hilo notifique a otros hilos sobre la ocurrencia de un suceso, evitando la necesidad de un sondeo constante (polling) por parte de los hilos que esperan. Esto mejora la eficiencia y reduce el consumo de recursos al evitar que los hilos "giren" esperando una condición [1], [6].

Tipos de Problemas que se Evitan al Usar Mutex, Barreras o Semáforos

Los mecanismos de sincronización como mutex, barreras y semáforos son fundamentales para gestionar la concurrencia y evitar una serie de problemas comunes en sistemas operativos y programación concurrente [3], [4], [5], [6]:

- **Condiciones de Carrera (Race Conditions):** Ocurren cuando múltiples procesos o hilos acceden y manipulan datos compartidos simultáneamente, y el resultado final depende del orden de ejecución. Los mutex y semáforos son las herramientas principales para prevenir esto, asegurando que solo un hilo acceda a la sección crítica a la vez.
- **Interbloqueos (Deadlocks):** Situación en la que dos o más procesos o hilos están bloqueados indefinidamente, esperando un recurso que está siendo retenido por otro proceso o hilo en el mismo conjunto. Aunque los mecanismos de sincronización son necesarios, un uso incorrecto de ellos (por ejemplo, un mutex no liberado) puede causar interbloqueos. La gestión cuidadosa de la adquisición y liberación de recursos es crucial para evitarlos.
- **Inanición (Starvation):** Sucede cuando un proceso o hilo es repetidamente ignorado por el programador o no puede obtener el recurso que necesita, incluso si el recurso está disponible en algún momento. Esto puede ocurrir con políticas de programación injustas o con un diseño deficiente de los mecanismos de sincronización. Los semáforos y mutex deben implementarse de manera que garanticen la equidad en el acceso a los recursos.
- **Inconsistencia de Datos:** Directamente relacionado con las condiciones de carrera. Si los datos compartidos no se protegen adecuadamente, las actualizaciones parciales o incorrectas pueden llevar a un estado inconsistente del sistema. Los mutex y semáforos garantizan la atomicidad de las operaciones en secciones críticas, manteniendo la coherencia de los datos.
- **Violaciones de Exclusión Mutua:** El objetivo principal de los mutex es asegurar que solo un proceso o hilo esté en su sección crítica en un momento dado. Sin estos mecanismos, esta propiedad fundamental de la concurrencia se violaría, llevando a resultados impredecibles.
- **Sincronización Incorrecta de Fases:** En algoritmos paralelos que requieren que todos los participantes completen una etapa antes de pasar a la siguiente, la falta de sincronización puede llevar a errores lógicos. Las barreras resuelven este problema, asegurando que todos los hilos esperen en un punto común.
- **Desperdicio de Recursos por Sondeo (Busy Waiting):** Cuando un hilo espera activamente una condición revisando repetidamente una variable compartida, consume ciclos de CPU innecesariamente. Los eventos y semáforos permiten que los hilos se bloqueen y sean despertados solo cuando la condición que esperan se cumple, evitando el sondeo ocupado.

Conclusiones

La concurrencia es una característica esencial de los sistemas operativos modernos, pero su gestión efectiva requiere el uso de mecanismos de sincronización robustos. Los mutex, las barreras y los eventos, como se ilustra con los ejemplos de Python [threading](#), son herramientas poderosas que permiten a los desarrolladores controlar el acceso a recursos compartidos, coordinar la ejecución de hilos y prevenir problemas críticos como las condiciones de carrera, los interbloqueos y la inconsistencia de datos. Comprender y aplicar correctamente estos mecanismos es fundamental para construir aplicaciones concurrentes fiables y eficientes.

Referencias

[1] Python Software Foundation. (2025, Noviembre 1). threading — Paralelismo basado en hilos. <https://docs.python.org/es/3.9/library/threading.html> [2] Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson. <https://www.pearson.com/us/higher-education/program/Stallings-Operating-Systems-Internals-and-Design-Principles-9th-Edition/PGM208068.html> [3] GeeksforGeeks. (2025, Julio 15). Concurrency in Operating System. <https://www.geeksforgeeks.org/operating-systems/concurrency-in-operating-system/> [4] Studocu. (n.d.). Concurrencia y Exclusión Mutua en Sistemas Linux. <https://www.studocu.com/latam/document/universidad-de-costarica/sistemas-operativos-i/concurrencias-y-exclusion-mutuas/63157954> [5] LibreTexts Engineering. (2021, Marzo 8). 5.1: Introduction to Concurrency. https://eng.libretexts.org/Courses/Delta_College/Operating_System%3A_The_Basics/05%3A_Process_Synchronization/5.1%3A_Introduction_to_Concurrency [6] Tutorialspoint. (2023, Abril 4). Concurrency in Operating System. <https://www.tutorialspoint.com/concurrency-in-operating-system>